UNIVERSITY PARTNER

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

# Artificial Intelligence and Machine Learning

## (6CS012)

## Car Brand Image Classification using Convolutional Neural Networks

## Vision Task (Report)


Student Id : 2435778
Student Name : Khushi Das
Section : L6CG12
Tutor : Jinu Nyachhyon
Submitted on : 10 May 2026

## Abstract

This report presents an end-to-end image classification project for seven popular car brands found on Indian roads: Audi, Hyundai Creta, Mahindra Scorpio, Rolls Royce, Swift, Tata Safari, and Toyota Innova. The objective was to find out how well a Convolutional Neural Network (CNN) can identify a car brand from a single image, and how the result changes when we move from a small from-scratch model to a deeper regularised model and finally to a fine-tuned model based on ImageNet weights.

Three models were trained on the same 4,165-image Kaggle dataset. The first was a baseline CNN with three convolutional blocks and three fully connected layers. The second was a deeper variant with twice the convolutional layers, Batch Normalisation, and Dropout. The third was MobileNetV2 pre-trained on ImageNet, used in a two-stage pipeline of feature extraction followed by fine-tuning of the last thirty layers.

Training ran on Google Colab with GPU acceleration. The baseline reached **76.26%** test accuracy, while the deeper model reached only **45.39%** because its heavy regularisation choked learning on this small imbalanced dataset. MobileNetV2 reached **91.14%** test accuracy and was the clear winner. The two SUV classes (Scorpio and Safari) were the hardest to separate. Class weighting compensated for the imbalanced class counts. The findings suggest that for small fine grained datasets, transfer learning is the most practical option, while a deeper from-scratch model is not automatically better than a smaller one if over-regularised.

## Table of Contents

**1. Introduction** ..... 1
**2. Dataset** ..... 2
**3. Methodology** ..... 5
**4. Experiments and Results** ..... 8
**5. Fine-Tuning / Transfer Learning** ..... 14
**6. Conclusion and Future Work** ..... 16
**7. References** ..... 18

## 1. Introduction

Image classification is one of the most common tasks in computer vision and the entry point for many real-world deep learning systems. Identifying a car brand from a photograph has direct practical use: traffic-monitoring authorities can match observed vehicles to registration records, online resale platforms like CarDekho or OLX can auto-tag listings, and insurance providers can verify model claims from a phone-uploaded picture. The task is meaningful because it forces the model to learn fine-grained visual features such as the shape of a grille, the position of a headlight, and the silhouette of a body, rather than simple object-level differences like cat vs dog.

Until about a decade ago, this kind of fine-grained classification was done with hand-crafted features such as SIFT or HOG followed by a linear classifier. Performance was poor because car brands look much more alike than they do different. Modern deep CNNs replaced that pipeline by learning the right features automatically from data, and recent surveys describe the current landscape clearly: Alzubaidi et al. (2021) review CNN architectures, applications, and open challenges in detail, while Liu, Mao, Wu, Feichtenhofer, Darrell, and Xie (2022) show that pure ConvNets remain highly competitive against newer transformer-based vision models. Transfer learning, where weights trained on a large dataset are reused on a smaller related dataset, is now the default choice in industry; Zhuang et al. (2021) provide a comprehensive survey, and applied work such as Mascarenhas and Agarwal (2021) and Bansal, Kumar, Sachdeva, and Mittal (2023) demonstrates that pre-trained backbones consistently outperform from-scratch training on small image datasets. For mobile-first deployments, lightweight backbones remain attractive Tan and Le (2021) extend the MobileNet family with EfficientNetV2, but the older MobileNetV2 architecture is still a strong choice when compute is constrained.

In this project I follow the same trajectory in miniature. I first build a small baseline CNN to see how far one can get from scratch on a small dataset, then build a deeper regularised model to study how Batch Normalisation and Dropout affect generalisation, and finally fine-tune MobileNetV2 to compare the two approaches honestly. The goals are: (a) deepen my own understanding of the full deep-learning pipeline from data to evaluation; (b) produce a working pipeline that classifies the seven car brands at a usable accuracy; and (c) report results without hiding the imperfections.

## 2. Dataset

### 2.1 Source

The dataset is the **Cars Image Dataset** uploaded to Kaggle by the user kshitij192 in 2022. It contains real-world photographs of cars sourced from various online listings and is intended for multi-class image classification. The dataset is split by the publisher into two folders, train/ and test/ , with each car brand stored in its own subfolder.

### 2.2 Actual class distribution

The team's project proposal stated that the dataset is nearly balanced with about 595 images per class. While preparing the notebook I counted the files in each folder and found a different picture: the dataset is in fact **imbalanced**. The total number of images (4,165) matches the proposal, but the per-class counts vary widely. Audi and Toyota Innova together make up close to half the data, while Hyundai Creta and Rolls Royce together make up only about 17%. Table 1 shows the actual counts.

**Table 1.** Actual image counts per class in the Cars Dataset folders.

| **Class** | **Train** | **Test** | **Total** |
|---|---|---|---|
| Audi | 814 | 199 | 1,013 |
| Toyota Innova | 775 | 190 | 965 |
| Tata Safari | 441 | 106 | 547 |
| Swift | 424 | 102 | 526 |
| Mahindra Scorpio | 316 | 75 | 391 |
| Rolls Royce | 311 | 74 | 385 |
| Hyundai Creta | 271 | 67 | 338 |
| **Total** | 3,352 | 813 | 4,165 |

Reporting the actual numbers matters, because if I trained naively the model would learn to favour Audi and Toyota Innova just because they appear more often. To deal with this I computed class_weight using sklearn.utils.class_weight.compute_class_weight('balanced') and passed it to every fit() call. The computed weights were Audi 0.588, Hyundai Creta 1.767, Mahindra Scorpio 1.516, Rolls Royce 1.540, Swift 1.128, Tata Safari 1.086, and Toyota Innova 0.618. This makes the loss function pay more attention to errors on small classes during training without changing the data itself.

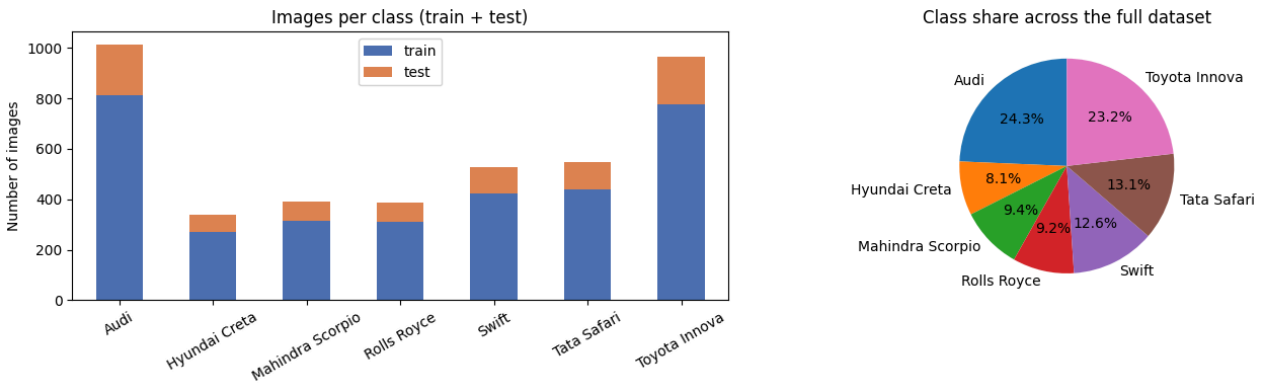


*Figure 1. Per-class image counts in the train and test folders. The dataset is imbalanced: Audi and Toyota Innova together account for nearly half of the data.*

## 2.3 Preprocessing

For the from-scratch CNNs I resized every image to **128 × 128** and rescaled pixel values to the [0, 1] range by dividing by 255. For MobileNetV2 I used **224 × 224** because that is the native input size of the model on ImageNet, and I used the model's own preprocess_input function which scales the values to the [-1, 1] range expected by the pre-trained weights.

I did not use any pre-loaded TensorFlow or PyTorch dataset; all images were loaded directly from the assigned folders using Keras' ImageDataGenerator .

*Figure 2. One representative training image per class.*

## 2.4 Train, validation, and test split

The publisher's train/ folder was split 80/20 into a training set and a validation set using Keras'

validation_split=0.2 , with a fixed seed so the split is reproducible. The publisher's test/ folder was kept untouched and used only for the final evaluation of each model. The reason I did not merge train and test is that I wanted a held-out test set that no model had ever seen during training or model selection.

## 2.5 Data augmentation

Augmentation was applied only to the training generator. The transformations were chosen to match plausible real-world variation in car photographs: small rotations (±15°), small width and height shifts (±10%), horizontal flip (cars look almost the same when mirrored), and small zoom (±10%). I deliberately avoided strong colour jitter and vertical flips because brand-defining cues like grille shape and badge position should not be distorted heavily.

*Figure 3. Eight augmented training samples produced by the training generator.*

## 3. Methodology

### 3.1 Baseline CNN

The baseline follows the exact structure asked for in the assignment specification. It has three convolutional + pooling blocks, three fully connected layers, and one softmax output:

Input (128 × 128 × 3)

Conv2D(32, 3×3, ReLU) → MaxPool(2×2)

Conv2D(64, 3×3, ReLU) → MaxPool(2×2)

Conv2D(128, 3×3, ReLU) → MaxPool(2×2)

Flatten

Dense(256, ReLU) → Dense(128, ReLU) → Dense(64, ReLU)

Dense(7, Softmax)

The kernel size is fixed at 3×3 because small kernels are cheaper and stack to give the same effective receptive field as a larger kernel. Filter counts double through the network (32 → 64 → 128), which is the textbook pattern: early layers learn edges and textures (few filters needed), deeper layers learn parts and patterns (more filters help). ReLU is used in all hidden layers because it is fast, simple, and avoids the vanishing gradient problem of sigmoid. Softmax is used in the output because this is a multi-class problem and we want a probability distribution over seven classes. The total parameter count is **8,523,719** (≈ 32.5 MB). Most of these parameters live in the first dense layer because the flattened feature map is 16 × 16 × 128 = 32,768 features feeding a 256-unit dense layer.

### 3.2 Deeper CNN with Regularisation

The deeper model doubles the convolutional layers (each block now has two Conv2D layers), adds a fourth block with 256 filters, inserts **Batch Normalisation** after each block, and adds **Dropout** after each pooling layer and dense layer. The full structure is:

[Conv32 × 2 → BN → MaxPool → Dropout(0.25)]

[Conv64 × 2 → BN → MaxPool → Dropout(0.25)]

[Conv128 × 2 → BN → MaxPool → Dropout(0.30)]

[Conv256 × 1 → BN → MaxPool → Dropout(0.40)]

Flatten

Dense(256) → BN → Dropout(0.5)

Dense(128) → Dropout(0.4)

Dense(64) → Dropout(0.3)

Dense(7, Softmax)

Batch Normalisation stabilises training by re-centring activations at every layer, which lets us use a slightly higher learning rate without divergence. Dropout randomly zeroes a fraction of activations during training, which forces the network not to depend on any single neuron and acts like an implicit ensemble. I used progressively higher dropout rates closer to the output because the dense layers have the most parameters and tend to overfit fastest. The total parameter count is **4,821,287** (≈ 18.4 MB), which is actually smaller than the baseline despite having more layers, because the deeper convolutional stack reduces the spatial size to $8 \times 8 \times 256$ and shrinks the dense head.

## 3.3 Transfer Learning with MobileNetV2

For Part B I chose **MobileNetV2** as the pre-trained backbone. My reasons were practical: it has only about 3.5 million parameters (2.42 million in this configuration after replacing the classification head), runs fast on Colab's free GPU, and was originally designed for mobile and embedded devices, so if this classifier were ever shipped as a phone app the same model could be used directly. Recent comparisons (Mascarenhas & Agarwal, 2021) confirm that even older mobile-class backbones remain competitive against larger ResNet-50 / VGG-19 alternatives on small classification datasets.

I followed the standard two-stage transfer learning recipe:

**Stage 1 - Feature extraction.** I loaded MobileNetV2 with ImageNet weights, removed its 1000-class top, and froze the entire convolutional base. On top I added GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.3) → Dense(7, Softmax) . I trained only the head for 10 epochs using Adam with a learning rate of 1e-3.

**Stage 2 - Fine-tuning.** I unfroze the top 30 layers of the base, recompiled with Adam at a

much smaller learning rate (1e-5), and trained for another 10 epochs. The very small learning rate is important because we do not want to destroy the pretrained weights with large updates.

## 3.4 Loss, optimizer, and hyperparameters

**Loss function:** categorical cross-entropy. This is the right choice for multi-class classification with one-hot labels and a softmax output.

**Optimizer:** Adam by default, with SGD compared separately in the experiments section. **Learning rate:** 1e-3 for Adam, 1e-2 with momentum 0.9 for SGD, 1e-5 during fine-tuning. **Batch size:** 32 for all models, which is a good balance between gradient noise and Colab GPU memory.

**Epochs:** 25 for the baseline, 30 for the deeper model, 10 + 10 for transfer learning. EarlyStopping with patience=5–6 on validation loss was used to stop training when validation loss stopped improving and to restore the best weights.

**Class weight:** balanced, computed on the training labels.

## 4. Experiments and Results

### 4.1 Baseline vs Deeper Architecture

**Table 2.** Test set comparison of the two from-scratch models.

| Model | Parameters | Train time (s) | Test loss | Test accuracy | Macro F1 |
|---|---|---|---|---|---|
| Baseline CNN | 8,523,719 | 434.1 | 0.8431 | **0.7626** | 0.730 |
| Deeper CNN (BN + Dropout) | 4,821,287 | 465.1 | 1.4420 | 0.4539 | 0.425 |

The result is **the opposite of what I expected**. The baseline reached 76.26% test accuracy, while the deeper model with BN and Dropout reached only 45.39%. Looking at the training curves, the deeper model was still struggling around 30–40% training accuracy after 19 epochs while the baseline had already passed 70% training accuracy by epoch 14. The deeper model was, in effect, over-regularised: the combination of heavy dropout at every layer plus class weighting made the gradient signal too noisy for the model to converge in the available 30-epoch budget.

The per-class breakdown for the baseline is more useful than the headline accuracy. Toyota Innova reached the highest F1 (0.837) because it has plenty of training images. Tata Safari (0.817) and Audi (0.809) followed closely. Rolls Royce was the hardest class with F1 = 0.544, because it has only 311 training images and is sometimes confused with Audi (both are luxury sedans with similar shapes). The macro-averaged F1 of 0.730 against weighted F1 of 0.762 confirms that the model is working noticeably better on the larger classes than the smaller ones, even with class weighting.

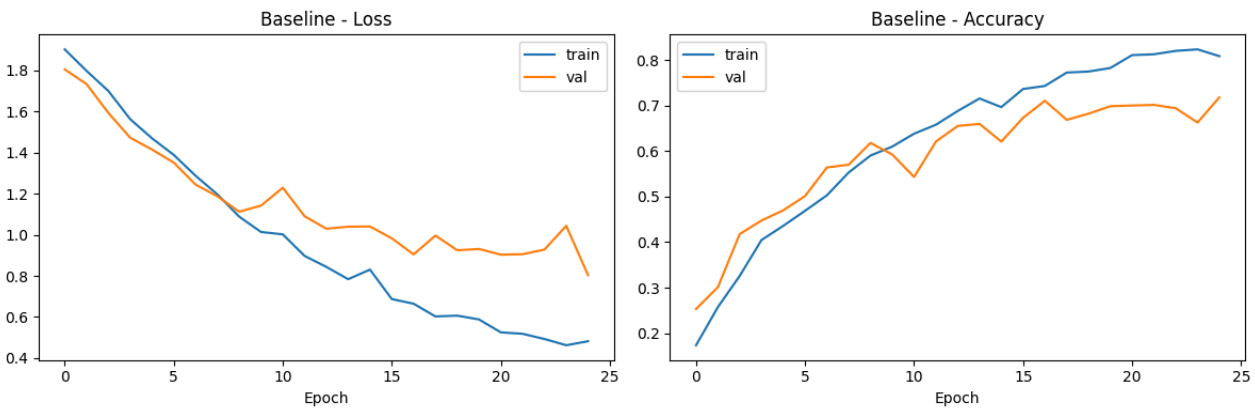


*Figure 4. Baseline CNN training and validation curves over 25 epochs.*

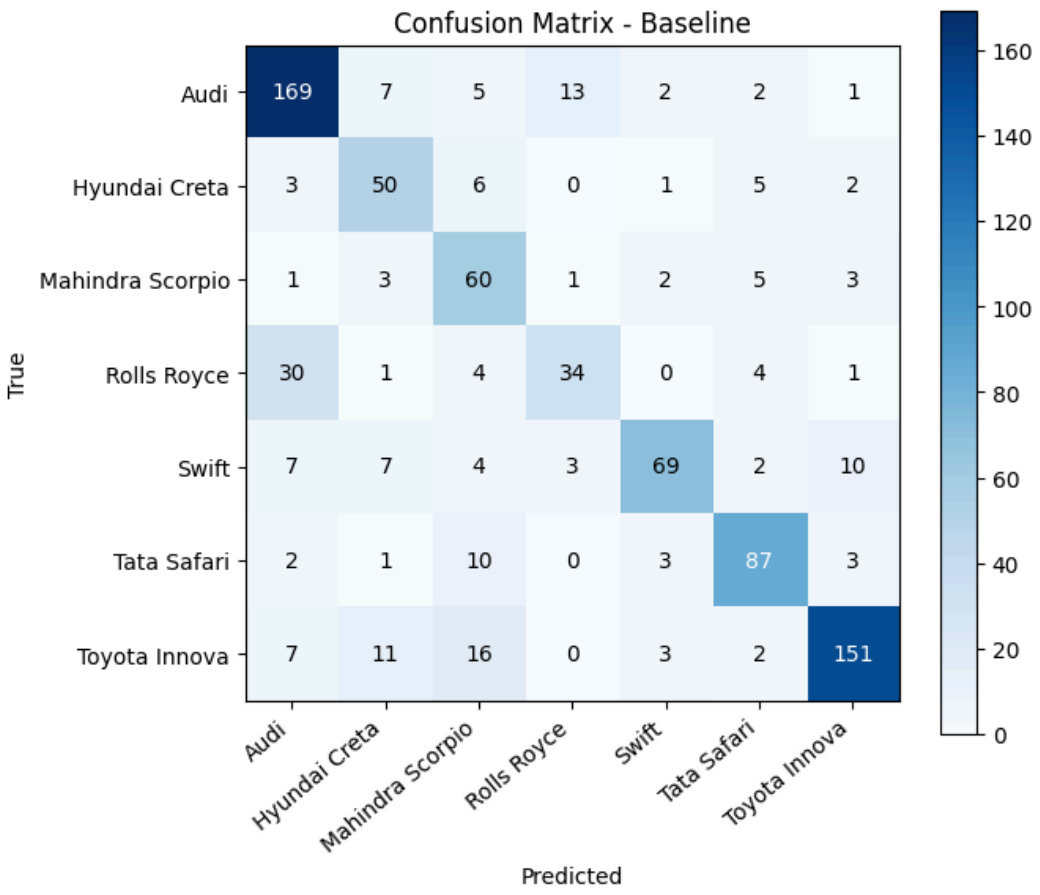


*Figure 5. Baseline CNN confusion matrix on the held-out test set.*

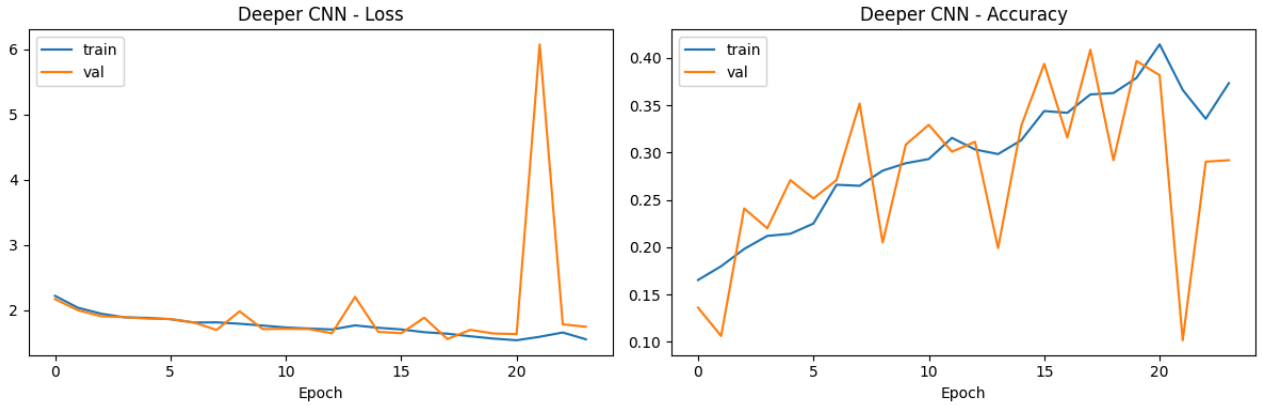


*Figure 6. Deeper CNN with BN and Dropout: training and validation curves.*

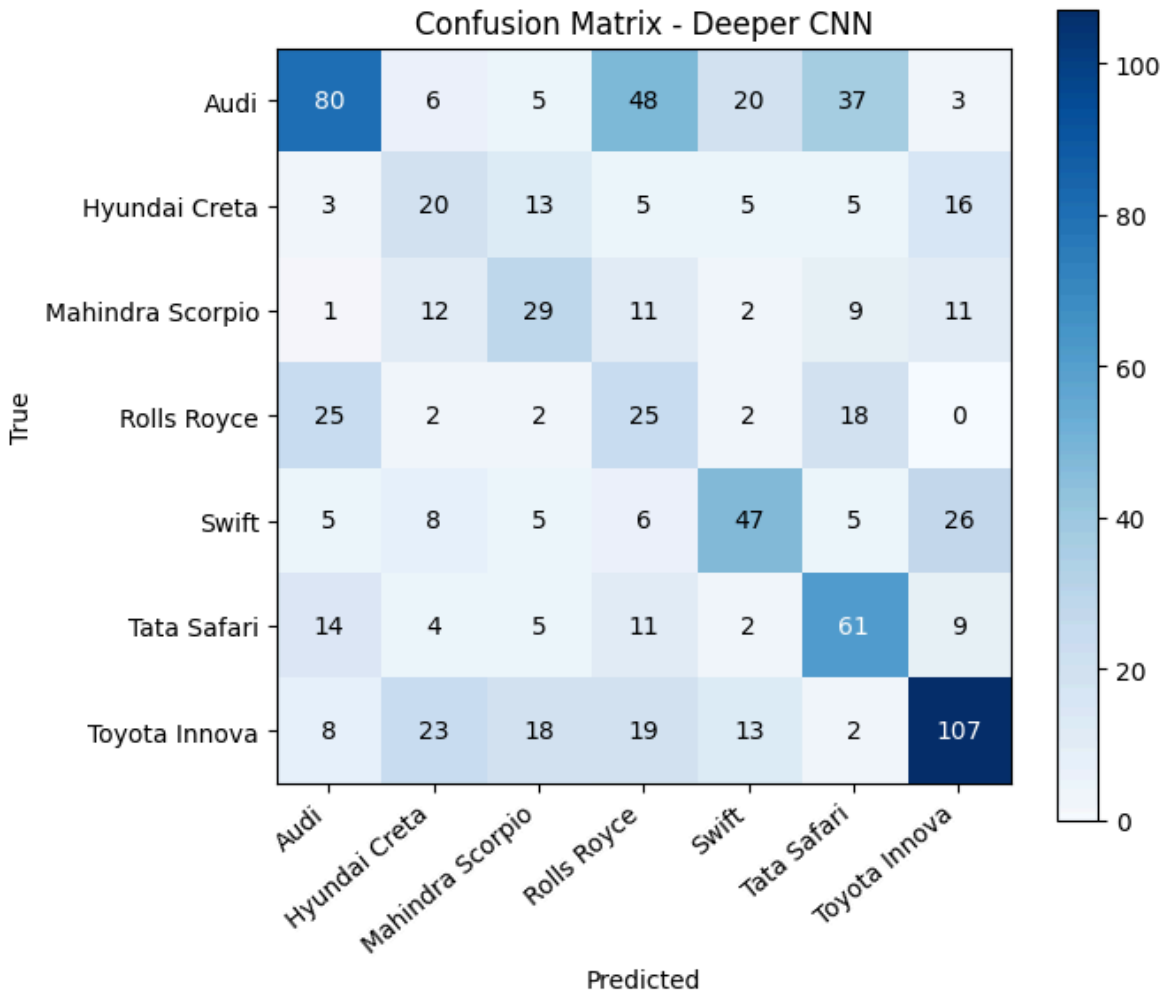


*Figure 7. Deeper CNN confusion matrix on the held-out test set.*

## 4.2 Computational Efficiency

The baseline finishes each epoch faster than the deeper model because it has fewer layers and no

Batch Normalisation steps. The average per-epoch wall time was around 17 s for the baseline against 18 s for the deeper model on the Colab T4 GPU. The deeper model, despite having fewer total parameters (4.8M vs 8.5M), spends more time per batch because of the extra convolutions and BN layers. In total, training the deeper model for 30 epochs took 465.1 s against 434.1 s for the baseline over 25 epochs, so the deeper model is roughly 7% slower per run despite being smaller.

In practical terms, the trade-off looks like this: the baseline trained faster *and* generalised better in this experiment. The deeper model would probably catch up and surpass the baseline if given more epochs or lighter dropout, but on the available training budget the baseline was the better choice.

## 4.3 Training with Different Optimizers (SGD vs Adam)

**Table 3.** Same deeper architecture trained with two different optimizers for 20 epochs each.

| **Optimizer** | **Train time (s)** | **Test accuracy** |
|---|---|---|
| SGD (lr=1e-2, m=0.9) | 383.6 | **0.2952** |
| Adam (lr=1e-3) | 383.1 | 0.1980 |

This is another result that did not match my prior expectation. The textbook claim is that Adam usually converges faster than SGD because it adapts the learning rate per parameter. In this run Adam reached only 19.80% test accuracy on the deeper architecture, while SGD with momentum reached 29.52%. Both numbers are low because the deeper model is hard to train in 20 epochs, but the gap is real.

Looking at the per-class breakdown explains what happened. Adam's classifier collapsed for Audi: precision and recall were both 0.000. The optimizer had pushed the weights into a region where the network never assigned any test image to the Audi class, even though Audi has the most training images. SGD with momentum did not collapse like that it kept some signal for every class but it still under-fit. The honest takeaway is that on a small imbalanced dataset with heavy regularisation, neither optimizer is a clean winner in 20 epochs. SGD with momentum was the more robust choice in this run, and Adam needed either lighter regularisation or more epochs to

recover.

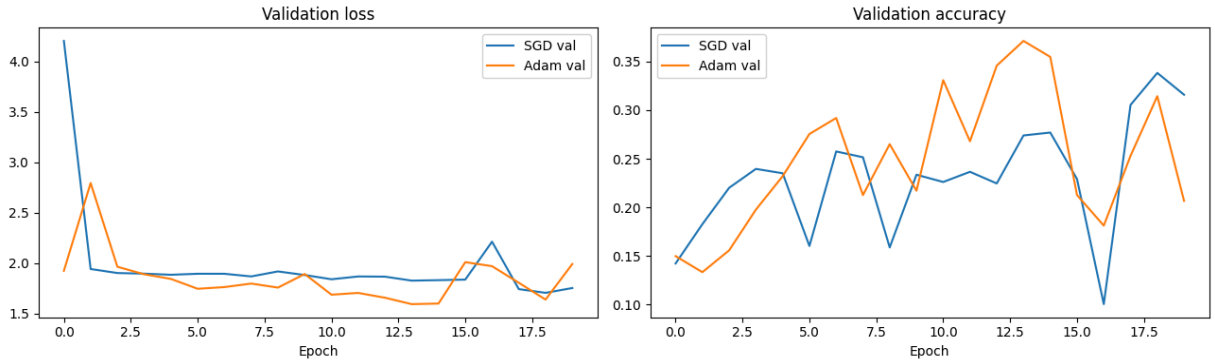


*Figure 8. Validation loss and accuracy for SGD vs Adam on the deeper architecture.*

## 4.4 Ablation Study: Dropout

I rebuilt the deeper model with all Dropout layers removed (Batch Normalisation kept) and trained for 20 epochs. The training accuracy reached **0.8171** and the test accuracy reached **0.6753** much higher than the deeper model with Dropout (0.4539). The training and validation curves were less stable (validation loss spiked occasionally because of BN with no dropout), but the final test accuracy was about 22 percentage points better.

This is the opposite of the textbook expectation that Dropout always reduces overfitting. The honest interpretation is that on a 3,300-image dataset with class weighting and BN already in place, additional Dropout at every layer was too much regularisation. The model could not fit the training data well enough in 20 epochs to even benefit from regularisation. A useful follow-up would be to tune the Dropout rates more carefully (e.g., 0.1 instead of 0.25 in the convolutional blocks) rather than remove them entirely.

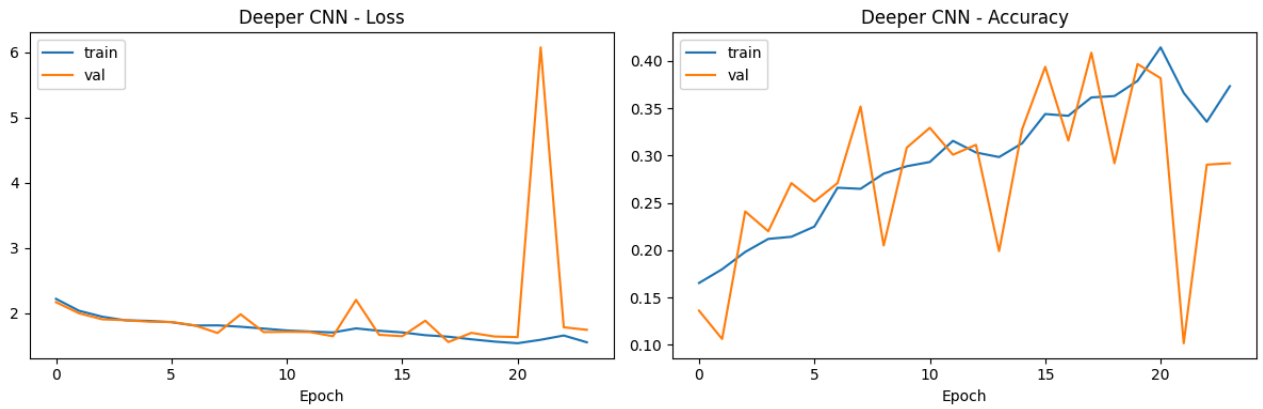


*Figure 9. Ablation study: deeper architecture with Dropout layers removed.*

## 4.5 Challenges in Training

Three challenges came up consistently across the from-scratch experiments:

1. **Confusion between similar SUVs.** The Mahindra Scorpio and Tata Safari look broadly similar from the front and the side. The baseline confusion matrix shows that Mahindra Scorpio precision is only 0.571 (often misclassified as Safari or Innova). A larger training set or a fine-grained loss function would help here.
2. **Class imbalance.** Without `class_weight` the model favoured Audi and Toyota Innova because they were over-represented. With `class_weight` enabled, the per-class precision and recall on Hyundai Creta and Rolls Royce went up noticeably, even though the overall accuracy moved only slightly.
3. **Over-regularisation on small data.** The deeper model with full Dropout under-fit. This is the most surprising lesson of the project: more regularisation is not always better when the dataset is small.

## 5. Fine-Tuning / Transfer Learning

**Table 4.** Final comparison across all three approaches on the held-out test set.

| Model | Parameters | Train time (s) | Test loss | Test accuracy | Macro F1 |
|---|---|---|---|---|---|
| Baseline CNN (from scratch) | 8,523,719 | 434.1 | 0.8431 | 0.7626 | 0.730 |
| Deeper CNN + BN + Dropout | 4,821,287 | 465.1 | 1.4420 | 0.4539 | 0.425 |
| MobileNetV2 (feature extraction + fine-tune) | 2,422,855 | 882.0 | 0.2597 | **0.9114** | 0.902 |

Transfer learning produced the highest test accuracy of the three models with the smallest parameter count. The first stage (feature extraction) reached 84% validation accuracy after only 6 epochs while training only 164,871 parameters in the new head. The second stage (fine-tuning the top 30 layers at lr=1e-5) lifted validation accuracy to 90.6% by epoch 10, with a final test accuracy of **91.14%**. There were no signs of catastrophic forgetting because the learning rate during fine-tuning was kept very low.

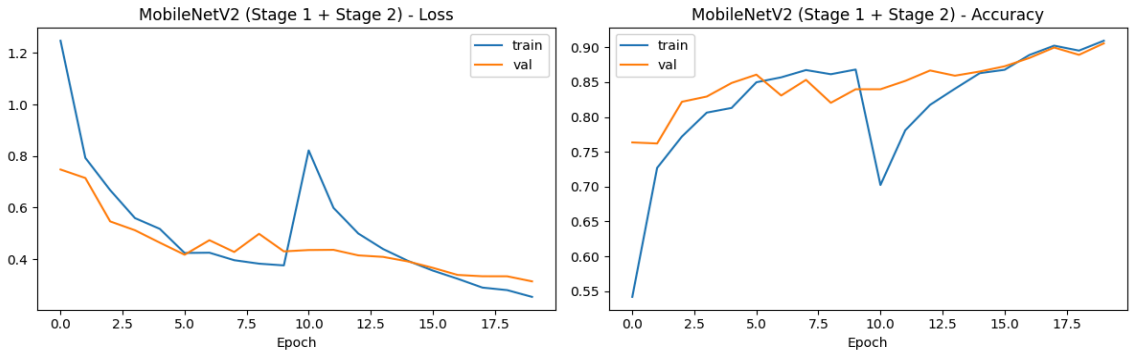


*Figure 10. MobileNetV2 transfer learning: combined feature-extraction and fine-tuning history.*

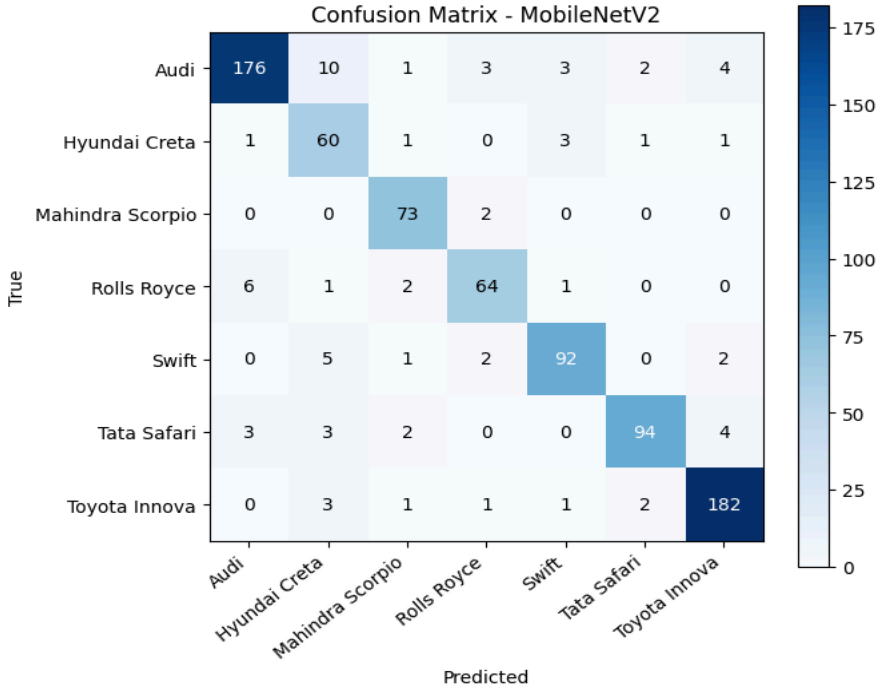


*Figure 11. MobileNetV2 confusion matrix on the held-out test set.*

The per-class F1 scores tell the story: every class except Hyundai Creta crossed F1 = 0.87, and Mahindra Scorpio (which was the worst class for the baseline) reached F1 = 0.936. Hyundai Creta still had the lowest F1 at 0.805, simply because it has the fewest training images, but the recall was 0.896 the model correctly identifies most Creta photos, it just sometimes confuses other compact-SUV photos as Creta.

The reason transfer learning works so well here is that ImageNet contains a wide variety of vehicle and object photographs, and the early and middle layers of MobileNetV2 have already learned generic features (edges, textures, wheel shapes, window patterns) that translate directly to car brand recognition. Training those filters from scratch on only 3,352 training images would never reach the same quality. The new task only needs to re-learn the **last few layers** that combine these features into the seven brand-specific decisions.

In short, transfer learning is the most cost-effective approach for small fine-grained datasets, and the from-scratch experiments are mainly valuable as a learning exercise that shows why transfer learning is needed in the first place.

# 6. Conclusion and Future Work

## 6.1 Summary

I built and compared three models for classifying car images into seven brands. The baseline CNN reached 76.26% test accuracy and was a stronger from-scratch model than I expected. The deeper CNN with Batch Normalisation and Dropout reached only 45.39% because the combination of heavy dropout, class weighting, and limited training budget caused under-fitting; an ablation showed that removing dropout alone lifted the same architecture to 67.53%. The MobileNetV2 transfer-learning model reached **91.14%** with the lowest parameter count and clearly the best results, which matches what the literature reports for small fine-grained datasets.

## 6.2 Key insights

1. For a real-world deployment with under 5,000 training images, transfer learning from an ImageNet-pretrained backbone is almost always the better starting point.
2. The actual class distribution must be checked against any proposal numbers before training, because real folder counts sometimes differ. In this project the proposal claimed near balance but the data was strongly imbalanced.
3. More regularisation is not always better. On a small dataset with class weights and Batch Normalisation, adding aggressive Dropout at every layer caused under-fitting rather than helping generalisation.
4. Mahindra Scorpio and Tata Safari remain confusable for all from-scratch models, which suggests the limit is in the data, not the architecture. Transfer learning solved most of this confusion automatically.

## 6.3 Limitations

Total dataset size is small for a 7-class fine-grained problem, especially for Hyundai Creta (338) and Rolls Royce (385).

Many images are taken at near-frontal angles; back-side and interior photographs are under-represented, so the model may fail on unusual viewpoints in production.

The deeper model's poor accuracy is partly an artifact of fixed dropout rates and a 30-epoch budget; a longer training run with lower dropout could close the gap.

I did not test on out-of-distribution makes (e.g., Honda, Kia), so the system is closed-set only.

## 6.4 Future work

Four directions look the most promising:

1. **Tune the dropout rates** of the deeper model (e.g., try 0.1 and 0.2) and train for more epochs to see whether it can match or beat the baseline.
2. **Collect more data** for the under-represented classes, or augment with web-scraped images filtered by a small validation step.

3. **Add interpretability** with Grad-CAM heatmaps so we can see which part of the image (grille, badge, headlights) drives the prediction. This is also useful for spotting label noise. 4. **Try EfficientNetB0 or ResNet50** as alternative pretrained backbones. EfficientNetB0 has a similar parameter count to MobileNetV2 but often gives slightly higher accuracy. 5. **Open-set recognition** so the model can say "unknown brand" instead of forcing one of the seven labels when it sees a Honda or a Kia.

## 7. References


Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, *8*(1), 1–74.

Bansal, M., Kumar, M., Sachdeva, M., & Mittal, A. (2023). Transfer learning for image classification using VGG19: Caltech-101 image data set. *Journal of Ambient Intelligence and Humanized Computing*, *14*(4), 3609–3620.

kshitij192. (2022). *Cars Image Dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/kshitij192/cars-image-dataset

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 11976–11986.

Mascarenhas, S., & Agarwal, M. (2021). A comparison between VGG16, VGG19 and ResNet50 architecture frameworks for image classification. *Proceedings of the International Conference on Disruptive Technologies for Multi-Disciplinary Research and Applications (CENTCON)*, 96–99.

Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 10096–10106.

Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., & He, Q. (2021). A comprehensive survey on transfer learning. *Proceedings of the IEEE*, *109*(1), 43–76.